

Advanced Functions

*args, **kwargs, Lambda, Recursion, Closures & Decorators

Session 9

What We're Covering Today

- Flexible arguments — *args, **kwargs, and unpacking into a call
- Lambda functions — short, anonymous, one-expression logic
- Recursion — functions that call themselves, and when that’s the right tool
- Closures — functions that remember the scope they were born in
- Decorators — wrapping one function’s behavior around another
- Today's tasks — industry-style, do-it-yourself practice
- Two mini projects combining everything from this session



PART 1

Flexible Arguments

What happens when you don’t know how many values a function needs to accept?

The Problem With Fixed Parameters



Unknown number of inputs

- A normal function locks in exactly how many arguments it accepts — def total(a, b, c) only ever takes three
- But real functions often can’t know that in advance: a logger might get 1 message or 5, a total() might sum 2 numbers or 50
- Adding a, b, c, d, e… parameters "just in case" doesn’t scale, and callers still hit a wall eventually
- Python’s answer is *args and **kwargs — built-in ways to accept "as many as you give me"



*args — Any Number of Positional Arguments

- A single * before a parameter name collects every extra positional argument into a tuple
- "args" is just a convention — the * is what matters, not the name
- Inside the function, args behaves like any other tuple — loop over it, index it, len() it

```
def total(*args):
    print(type(args))    # <class 'tuple'>
    return sum(args)

total(4, 9)          # 13
total(1, 2, 3, 4, 5)    # 15
total()              # 0
```

**kwargs — Any Number of Keyword Arguments

- Two asterisks before a parameter name collect every extra keyword argument into a dictionary
- Each caller-supplied name becomes a key, and the value they passed becomes that key’s value
- This is exactly how flexible config, settings, and options are usually handled in real APIs

```
def build_profile(**kwargs):
    print(type(kwargs))   # <class 'dict'>
    return kwargs

build_profile(name="Ahmed", age=25)
# {'name': 'Ahmed', 'age': 25}

build_profile(city="Karachi")
# {'city': 'Karachi'}
```

Combining Them — and the Order Python Requires

- Regular parameters, *args, and **kwargs can all appear in one function — but only in this fixed order
- Regular parameters first, then *args, then **kwargs — Python enforces this at definition time
- Getting the order wrong is a SyntaxError, not a runtime bug — it’s caught before the code even runs

```
def order(item, *args, **kwargs):
    print(item)    # regular param
    print(args)    # extra positional
    print(kwargs)  # extra keyword

order("book", "pen", "bag", price=50, qty=2)
# book
# ('pen', 'bag')
# {'price': 50, 'qty': 2}
```

Unpacking — Spreading a List or Dict Into a Call

- * and ** also work in the other direction — spreading an existing list or dict out as separate arguments
- *my_list unpacks each item as its own positional argument, in order
- **my_dict unpacks each key as a keyword argument — the keys must match the function's parameter names

```
def greet(name, age):
    print(f"{name} is {age}")

values = ["Ahmed", 25]
greet(*values)        # Ahmed is 25

info = {"name": "Sara", "age": 22}
greet(**info)         # Sara is 22
```

Real-World Use: A Flexible Logging Function

- Production logging functions never know in advance how many pieces of context they'll be given
- *args lets the caller pass any number of message parts; **kwargs lets them attach any extra metadata
- This exact pattern — *args + **kwargs — is how most real logging, wrapper, and utility functions are built

```
def log(level, *parts, **meta):
    message = " ".join(str(p) for p in parts)
    tags = ", ".join(f"{k}={v}" for k, v in meta.items())
    print(f"[{level}] {message}  ({tags})")

log("INFO", "User", "logged in", user_id=42,
ip="10.0.0.1")
# [INFO] User logged in  (user_id=42, ip=10.0.0.1)
```

Practice Set — Flexible Arguments

Try these yourself first — no code shown, that's the point

1. Write a function multiply_all(*args) that multiplies every number passed to it

2. Write a function describe(**kwargs) that prints each key and value on its own line

3. Combine both: a function that takes a required name, then any number of hobbies, then any number of extra facts as keywords

4. Take an existing list of 3 numbers and unpack it into a function that expects three separate arguments



PART 2

Lambda Functions

Sometimes a function is so small it doesn’t deserve a name — or a def block

What Is a Lambda?



One line.
One value.
No name.

- A lambda is a small, anonymous function — defined and used in a single expression, with no def and no function name
- lambda x: x * 2 is a complete function — it takes x, and its result is whatever the expression evaluates to
- There's no return keyword — the expression's value is automatically returned
- Lambdas exist for one purpose: to hand a tiny, throwaway function to something else that expects one

Lambda vs. def — When Each Is the Right Choice

Use lambda when

- The logic is one simple expression
- You're passing it directly as an argument (to sorted(), map(), filter()…)
- It will never be reused or called by name elsewhere
- Naming it would add a line without adding clarity

Use def when

- The logic needs more than one expression, or any statements
- You'll call it more than once, from more than one place
- A descriptive name would make the code easier to read
- It needs a docstring, type hints, or error handling

Lambda + sorted() — A Perfect Pairing

- sorted() accepts a key= function that tells it what to sort by — exactly what lambda is built for
- Compare this to writing a full, separately-named function just to sort a dictionary by value
- For a one-off sort key, the lambda version is shorter and just as clear

```
scores = {"Ahmed": 88, "Sara": 95, "Ali": 72}

# Old way: a full named function
def by_value(pair):
    return pair[1]
sorted(scores.items(), key=by_value)

# With lambda — same result, one line
sorted(scores.items(), key=lambda pair: pair[1])
# [('Ali', 72), ('Ahmed', 88), ('Sara', 95)]
```



Lambda + map() and filter()

- map() applies a function to every item in a sequence and returns the transformed results
- filter() applies a function that returns True/False, and keeps only the items that pass
- Both expect a function as their first argument — lambda supplies it inline, without a separate def

```
nums = [1, 2, 3, 4, 5, 6]

doubled = list(map(lambda n: n * 2,
nums))
# [2, 4, 6, 8, 10, 12]

evens = list(filter(lambda n: n % 2 ==
0, nums))
# [2, 4, 6]
```

When NOT to Use Lambda



Readable code beats clever code

- If a lambda needs a comment to explain what it does, it should be a named function instead
- Chaining multiple lambdas together, or nesting them, is a common real-world code-review flag — it hurts readability fast
- If the same logic is used in more than one place, give it a name — a lambda can’t be reused cleanly or documented
- Professional teams treat "would this be clearer as a def?" as the actual test — not "can I fit it on one line?"

Practice Set — Lambda

Try these yourself first — no code shown, that's the point

1. Write a lambda that takes a number and returns whether it’s greater than 10

2. Use sorted() with a lambda to sort a list of names by their length

3. Use map() with a lambda to convert a list of Celsius temperatures to Fahrenheit

4. Use filter() with a lambda to keep only the words longer than 4 characters from a sentence



PART 3

Recursion

A function that solves a problem by asking a smaller version of itself to help

What Is Recursion?



A function calling itself

- Recursion is a function that calls itself to solve a smaller piece of the same problem
- Think of Russian nesting dolls — open one, and there’s a smaller identical doll inside, until you reach the smallest one that doesn’t open
- Each call works on a slightly smaller version of the problem, and the smallest version is simple enough to answer directly
- That "simple enough to answer directly" case is what makes the whole chain eventually stop



The Two Required Parts



Base Case

- The smallest version of the problem — answered directly, with no further recursive call
- This is what stops the recursion
- Every recursive function needs at least one
- Example: factorial(0) is defined to be 1, no calculation needed



Recursive Case

- The step where the function calls itself on a smaller piece of the problem
- Must always move toward the base case — never sideways, never backward
- Combines its own small piece of work with the result of the smaller call
- Example: factorial(n) = n × factorial(n - 1)

Classic Example: Factorial

- factorial(n) is n × (n-1) × (n-2) × … × 1 — a textbook recursive definition
- Base case: factorial(0) is 1
- Recursive case: factorial(n) is n times factorial(n - 1)

```
def factorial(n):
    if n == 0:          # base case
        return 1
    return n * factorial(n - 1)   # recursive case

factorial(4)
# 4 * factorial(3)
# 4 * (3 * factorial(2))
# 4 * (3 * (2 * factorial(1)))
# 4 * 3 * 2 * 1 * 1  ->  24
```

A Recursion Call Stack, Visualized



Each call waits for the one below it to finish — then the results multiply back up the stack: 1, then 2, then 6, then 24.

Classic Example: Fibonacci

- Each Fibonacci number is the sum of the two before it: 0, 1, 1, 2, 3, 5, 8, 13…
- Base cases: fib(0) is 0, fib(1) is 1
- Recursive case: fib(n) is fib(n-1) + fib(n-2) — two recursive calls, not one

```
def fib(n):
    if n <= 1:              # base case
        return n
    return fib(n-1) + fib(n-2)   # recursive case

fib(6)   # 8
[fib(n) for n in range(7)]
# [0, 1, 1, 2, 3, 5, 8]
```

The Danger of Infinite Recursion



No base case = no stop

- If a function never reaches its base case, it keeps calling itself — forever, in theory
- In practice, Python raises a RecursionError once the call stack hits its limit (by default, around 1000 calls deep)
- This is the recursive cousin of an infinite while loop — same root cause: the stopping condition is never satisfied
- The fix is always the same: check that every recursive call is moving strictly closer to the base case

Recursion vs. Loops — Honest Tradeoffs

Loops win when

- The repetition is simple and flat — no natural nesting
- Performance matters — loops avoid the overhead of repeated function calls
- The data is a straightforward sequence you're stepping through once

Recursion wins when

- The problem is naturally nested — trees, folders, nested data of unknown depth
- Each step is a smaller version of the exact same problem
- The recursive version is dramatically easier to read than the loop-based equivalent



Real-World Use: Nested Data of Unknown Depth

- A dictionary can contain another dictionary, which can contain another — to any depth, unknown in advance
- A loop can’t handle "unknown depth" cleanly, but recursion handles it naturally: dig in, and if you find another dictionary, recurse
- The exact same pattern is used to walk folder structures, JSON responses, and org charts

```
def sum_nested(data):
    total = 0
    for value in data.values():
        if isinstance(value, dict):
            total += sum_nested(value)   # recurse
        else:
            total += value
    return total

sum_nested({"a": 1, "b": {"c": 2, "d": {"e": 3}}})
# 6
```

Practice Set — Recursion

Try these yourself first — no code shown, that's the point

1. Write a recursive function that returns the sum of all numbers from 1 to n

2. Write a recursive function that reverses a string (no slicing shortcuts)

3. Trace factorial(5) by hand, writing out each call on paper before running any code

4. Write a recursive function that counts how many times a value appears in a nested list



PART 4

Closures

A function that remembers the variables from where it was created — even after that place is gone

What Is a Closure?



Remembers where it was born

- A closure is an inner function that "remembers" the variables from its enclosing function, even after that outer function has finished running
- Normally, a function’s local variables disappear once it returns — closures are the exception
- This happens automatically whenever an inner function uses a variable from the function that defines it, and that inner function is returned or passed elsewhere
- It’s what lets you build customized, reusable functions out of a single template function

A Closure, Visualized

make_multiplier(factor) — outer function, runs once

factor = 3

```
def multiply(n):
    return n * factor
```

↑ "factor" isn't local here — it reaches back into the outer function's scope and remembers it

double = make_multiplier(3) returns multiply, but multiply keeps its own private copy of factor = 3, even though make_multiplier has already finished running.



Real-World Use: A Function Factory

- make_multiplier(factor) returns a brand-new function, customized with whatever factor was passed in
- Each returned function keeps its own private copy of factor — double and triple don’t interfere with each other
- This "function that builds functions" pattern shows up constantly in real configuration and callback code

```
def make_multiplier(factor):
    def multiply(n):
        return n * factor    # remembers "factor"
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)

double(5)   # 10
triple(5)   # 15
```



nonlocal — Modifying an Enclosing Variable

- By default, an inner function can read an outer variable, but assigning to it creates a new local variable instead
- nonlocal tells Python: "this name refers to the enclosing function’s variable, not a new local one"
- This is how a closure keeps running state between calls — like a counter that remembers its own count

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

counter = make_counter()
counter()  # 1
counter()  # 2
counter()  # 3
```

Practice Set — Closures

Try these yourself first — no code shown, that's the point

1. Write a make_greeter(greeting) function that returns a function saying "{greeting}, {name}!" for any name

2. Write a make_power(exponent) factory — make_power(2) should return a function that squares its input

3. Build a counter with nonlocal that can count both up and down, using two inner functions sharing one variable



PART 5

Decorators

A function that wraps another function, adding behavior without touching its code

What Is a Decorator?



Wraps a function in extra behavior

- A decorator is a function that takes another function as input, and returns a new function that wraps it with extra behavior
- The original function usually still runs — the decorator just adds something before it, after it, or around it
- Think of it like gift-wrapping: the gift inside doesn't change, but wrapping paper adds something around it
- This is a light-touch introduction — you'll recognize decorators in real code long before you need to write your own

A Decorator, Visualized

wrapper() — what actually runs when you call the decorated function

```
print("Before the function runs")
    func()  — the original, un-modified function
print("After the function runs")
```

The original function sits untouched at the center — the decorator only adds layers around it, the same way wrapping paper adds nothing to the gift inside.

A Simple Working Example

- my_decorator takes a function, defines a wrapper() that adds behavior, and returns that wrapper
- The wrapper calls the original function in the middle — that's the "extra behavior wrapped around it" in action
- This before/after pattern is the foundation every real-world decorator builds on

```
def my_decorator(func):
    def wrapper():
        print("Before the function runs")
        func()
        print("After the function runs")
    return wrapper

def say_hello():
    print("Hello!")

decorated = my_decorator(say_hello)
decorated()
# Before the function runs
# Hello!
# After the function runs
```

The @ Syntax — Just Syntactic Sugar

- Writing @my_decorator directly above a function definition does exactly the same thing as the manual version on the last slide
- @my_decorator above def say_hello(): is Python's shorthand for say_hello = my_decorator(say_hello)
- Nothing new is happening — it's the same wrapping, just with cleaner, more readable syntax

```
def my_decorator(func):
    def wrapper():
        print("Before the function runs")
        func()
        print("After the function runs")
    return wrapper

@my_decorator        # same as: say_hello = my_decorator(say_hello)
def say_hello():
    print("Hello!")

say_hello()  # already wrapped — just call it directly
```



Where You’ll Actually See This



Flask Routes

@app.route("/home") turns an ordinary function into a URL endpoint — the most common decorator you’ll meet



Timing Decorators

A decorator that measures and prints how long a function took to run — without changing the function itself



Access Control

@login_required blocks a function from running unless the user is authenticated — used constantly in web apps

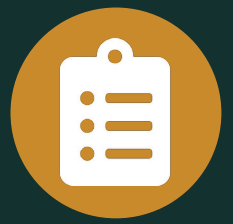
Practice Set — Decorators

Try these yourself first — no code shown, that's the point

1. Write a decorator that prints "Running..." before any function it wraps runs

2. Apply your decorator to two different functions using @ syntax

3. Predict, in your own words, what say_hello = my_decorator(say_hello) does — before checking your answer



INDUSTRY-STYLE TASKS

Build the Things You'll Actually Be Asked to Build

Not toy examples — flexible logging, safe recursion, and reusable factories are everyday, real-world work

Today's Tasks — Part A: Flexible Args & Lambda

Small tasks — do them in order

1. Build a flexible logging function log(level, *messages, **context) that joins the messages and prints each context key=value pair — this is the exact shape of real production logging calls.

2. You’re given a list of product dictionaries like [{"name": "Pen", "price": 20}, ...]. Use sorted() with a lambda to sort them by price, then by name.

3. Write a function apply_discount(prices, rate) that uses map() with a lambda to return every price reduced by rate (e.g. rate=0.1 for 10% off).

Today's Tasks — Part B: Recursion

Small tasks — do them in order

1. Write a recursive function count_down(n) that prints every number from n down to 1, then prints "Liftoff!" — with a correct base case.

2. Write a recursive function that sums every number in a nested list of unknown depth, e.g. [1, [2, 3, [4, 5]], 6] should return 21.

3. Write a recursive function flatten(data) that takes a dictionary of unknown nesting depth and returns a single flat list of all its values.

Today's Tasks — Part C: Closures & Decorators

Small tasks — do them in order

1. Build a make_validator(min_length) factory that returns a function checking whether a given password meets that minimum length.

2. Write a calculate_discount(price, pct) function, then rewrite it so an outer apply_tax(rate) closure wraps and adjusts its result.

3. Write a decorator that catches and reports repeated calls — print how many times the wrapped function has been called, each time it runs.



PART F

Put It All Together

Two projects combining flexible arguments, lambda, recursion, closures, and decorators into one cohesive task

Mini Project 1: Configurable Retry Decorator



Closure + decorator, combined

- Write a decorator factory retry(times) — called as @retry(3) — that re-runs the wrapped function up to that many times if it raises an exception
- retry(times) is itself a closure: it takes times, and returns the actual decorator that remembers it
- Use *args and **kwargs in the inner wrapper so it works with any wrapped function, no matter its own arguments
- This is genuinely how retry logic is built in real-world networking and API code

Mini Project 2: Recursive File-Tree Summary



Recursion + lambda, combined

- You’re given a nested dictionary representing folders and files, with unknown depth: {"src": {"main.py": 120, "utils": {"helpers.py": 45}}}
- Write a recursive function that returns the total line count across every file, at every depth
- Then write a second version that uses sorted() with a lambda to return the top 3 largest files by line count, across the whole tree
- This mirrors how real project-analysis and disk-usage tools are actually built



RECAP

*args and **kwargs make functions flexible. Lambda, recursion, closures, and decorators make them powerful.

Next session: Object-Oriented Programming — classes, objects, __init__, and inheritance